

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_8

8. Proxy Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Proxy pattern.

GoF Definition

It provides a surrogate or placeholder for another object to control access to it.

Concept

There are situations where you want to restrict direct communication between an intended object and the outside world. There are various reasons for this. For example, you may be promoting additional securities, you need to improve the response time, you may have some resource constraints, or the final product is still in the development phase.

Proxies can be of different types, but fundamentally they are substitutes (or placeholders) for an original object. As a result, when a client interacts with a proxy object, it appears that it is directly talking to the actual object. So, using this pattern, you may want to use a class that can perform as an interface to the original one.

Real-Life Example

Consider the case when a particular teacher in a school is absent due to some unavoidable circumstances. The school authority can send another teacher as a replacement to teach the classes. Consider an example from a different domain, say ATMs. An ATM implementation can hold proxy objects for bank information that actually exists on a remote server.

Computer World Example

In the programming world, creating multiple instances of a complex object can be costly. This is because you may need resources that are not easily available or allocatable. In such a situation, you can create multiple proxy objects that can point to the original object. This mechanism can help you to save computer/system memory and improve the performance of your application.

Another very common use of a proxy is when users do not want to disclose the true IP address of their machine; instead, they want to make it anonymous. Here are few more examples:

- In the `java.lang.reflect` package, the `Proxy` class and the `InvocationHandler` interface support a similar concept.
- The package `java.rmi.*` also provides methods through which an object in one Java virtual machine can invoke methods on an object that resides in a different Java virtual machine.
- You can see this pattern in other domains, too. For example, .NET developers often use a Windows Communication Foundation (WCF) framework for building service-oriented applications. There you may notice the use of WCF client proxies, which a client application uses to communicate with the service. You can also configure a REST API to work behind a proxy server to promote authorized communication.

Implementation

This chapter offers two demonstrations. In the first program, `Subject` is an abstract class with an abstract method called `doSomeWork()`. It has the following definition:

```
public abstract class Subject {  
    public abstract void doSomeWork();  
}
```

`ConcreteSubject` is a concrete class that inherits from `Subject` and provides the implementation for the `doSomeWork()` method. It looks like the following:

```
// The ConcreteSubject class  
class ConcreteSubject extends Subject {  
    ConcreteSubject(){}  
    @Override  
    public void doSomeWork() {
```

```
        System.out.println("The doSomeWork() is executed.");  
    }  
}
```

Let's assume you want to restrict the client from directly invoking the method in `ConcreteSubject`. (Consider the cases discussed in the computer world examples for the reasons behind this). So, you make a proxy class called `ProxySubject`. In your implementation, the `ProxySubject` class contains the same-named method `doSomeWork()` and a client can use this method using a `Proxy` instance. When a client calls the `doSomeWork()` method of the proxy object, the call propagates to the `doSomeWork()` method in the `ConcreteSubject` object. This gives the clients a feeling that they have invoked the method from the `ConcreteSubject` directly. This is why the `ProxySubject` class looks like the following:

```
/**  
 * This is the proxy class.  
 * It invokes the doSomeWork() from  
 * the ConcreteSubject.  
 */  
public class ProxySubject extends Subject {  
    private Subject subject;  
    public ProxySubject() {  
        // We create only one instance  
        // of the ConcreteSubject  
        if (subject == null) {  
            subject = new ConcreteSubject();  
        }  
    }  
    @Override  
    public void doSomeWork() {  
        System.out.println("Proxy call is happening now.");  
        subject.doSomeWork();  
    }  
}
```

Class Diagram

Figure [8-1](#) shows the class diagram.

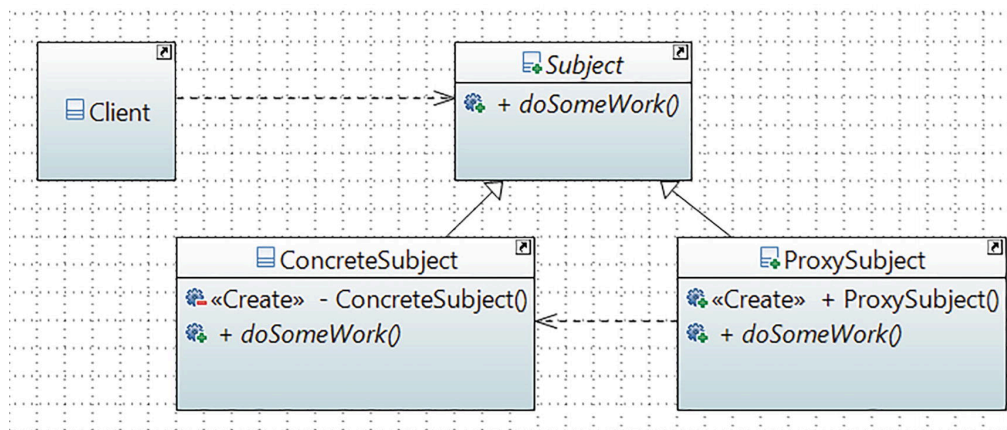


Figure 8-1 Class diagram

Package Explorer View

Figure **8-2** shows the high-level structure of the program.

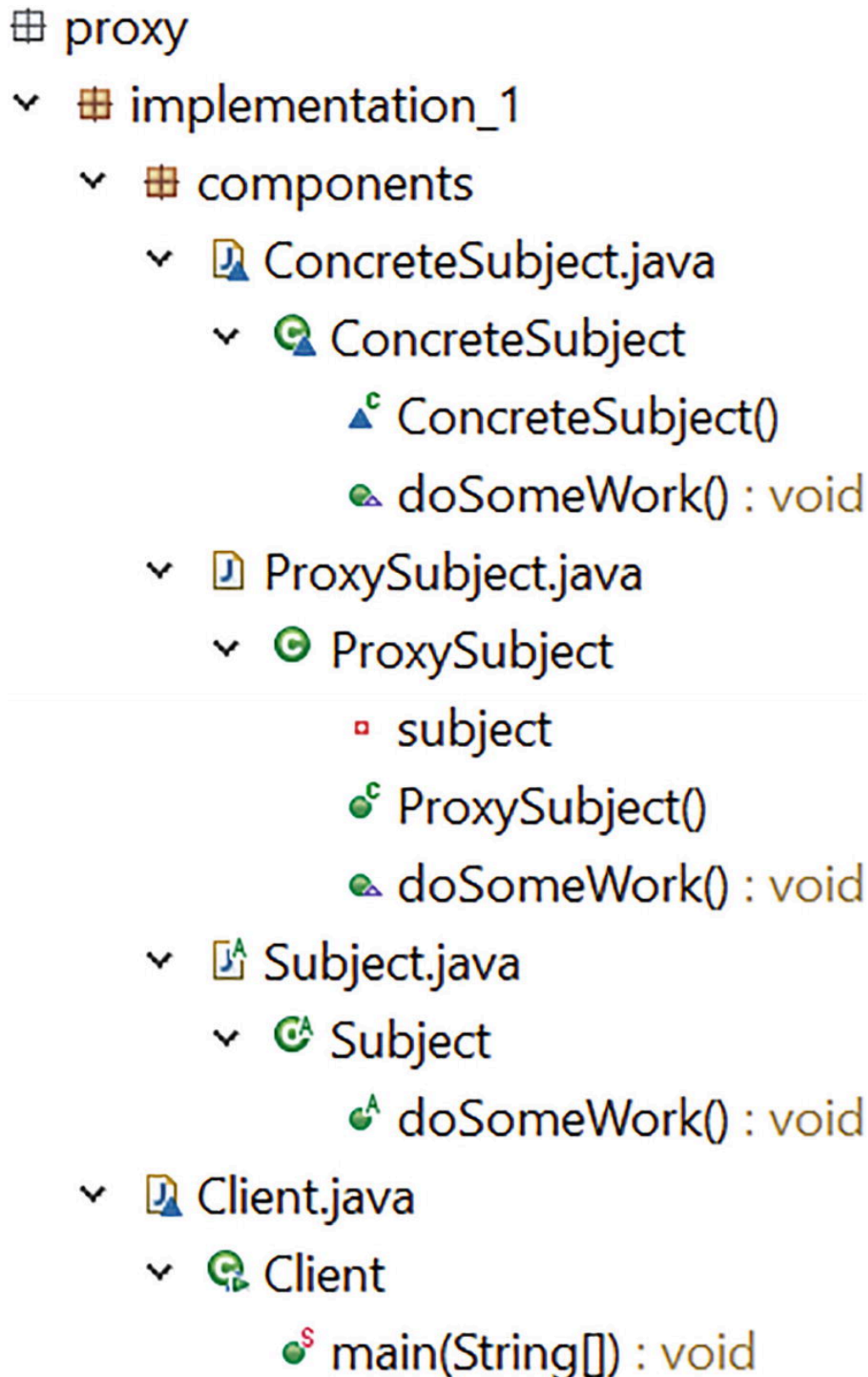


Figure 8-2 Package Explorer view

Demonstration 1

The client code is stored inside the package `jdp3e.proxy.implementation_1`. The remaining code is stored inside the package `jdp3e.proxy.implementation_1.components`.

Notice that the `ConcreteSubject` class and its constructor have package-private visibility. This is why inside the client code you cannot directly make an instance of `ConcreteSubject` so you are forced to use the `ProxySubject` class.

Lastly, there's only one instance of `ConcreteSubject`. If needed, you can reuse the same instance and avoid unnecessary object creation. This part is optional, but you know that it is a better practice. Here's the complete implementation:

```
// Subject.java
package jdp3e.proxy.implementation_1.components;
public abstract class Subject {
    public abstract void doSomeWork();
}

// ConcreteSubject.java
package jdp3e.proxy.implementation_1.components;
class ConcreteSubject extends Subject {
    ConcreteSubject(){}
    @Override
    public void doSomeWork() {
        System.out.println("The doSomeWork() is executed.");
    }
}

// ProxySubject.java
package jdp3e.proxy.implementation_1.components;
/**
 * This is the proxy class.
 * It invokes the doSomeWork() from
 * the ConcreteSubject.
 */
public class ProxySubject extends Subject {
    private Subject subject;
    public ProxySubject() {
        // We create only one instance
        // of the ConcreteSubject
        if (subject == null) {
            subject = new ConcreteSubject();
        }
    }
    @Override
    public void doSomeWork() {
        System.out.println("The proxy call is happening now.");
        subject.doSomeWork();
    }
}
```

```
// Client.java
package jdp3e.proxy;
import jdp3e.proxy.implementation_1components.*;
/**
 * The client is talking to a ConcreteSubject
 * instance through a proxy method.
 */
class Client {
    public static void main(String[] args) {
        System.out.println("***Proxy Pattern Demo***\n");
        // ConcreteSubject has package-private visibility
        // So, you cannot make an instance of it here.
        //Subject cs = new ConcreteSubject();//Error
        Subject proxy = new ProxySubject();
        proxy.doSomeWork();
    }
}
```

Output

Here's the output of the program:

```
***Proxy Pattern Demo***
The proxy call is happening now

.
The doSomeWork() is executed.
```

Q&A Session

8.1 What are the different types of proxies?

These are the common types of proxies:

- *Remote proxies*: These proxies can hide an object that sits in a different address space.
- *Virtual proxies*: These proxies are used to perform optimization techniques such as creating a heavy object on an on-demand basis.
- *Protection proxies*: These proxies generally deal with different access rights.

- *Smart reference*: These proxies can perform additional housekeeping when an object is accessed by a client. A typical operation may include counting the number of references to an object at a particular moment in time.

8.2 Why do you separate the client code from the rest of the code? Is it necessary?

It is a better practice in the sense that you do not allow a client to access the `ConcreteSubject` directly. This class and its constructor have package-private visibility. So, the clients are forced to use the Proxy class.

8.3 Using the lazy instantiation process, you may create unnecessary objects in a multithreaded application. Is this correct?

Yes. In this book, I present simple illustrations only, so I ignore that part. In the discussions of the Singleton pattern in Chapter 7, I analyze alternative approaches that tell you how to work in a multithreaded environment. You can always refer to those discussions in situations like this. (For example, in this particular scenario, you could implement a smart proxy to ensure that a particular object is locked before you grant access to the object).

8.4 Can you give an example of a remote proxy?

Suppose you want to call a method of an object, but the object is running in a different address space (for example, in a different location or on a different computer). How can you proceed? With the help of remote proxies, you can call the method on the proxy object, which in turn will forward the call to the actual object that is running on the remote machine. (You can refer to demonstration 1 as an example of this category in this context if the actual method exists on a different computer and you connect to it via a proxy object over a network). This type of need can be realized through different well-known mechanisms such as ASP.NET, CORBA, COM/DCOM, or Java's RMI.

Figure 8-3 shows a simple remote proxy structure.

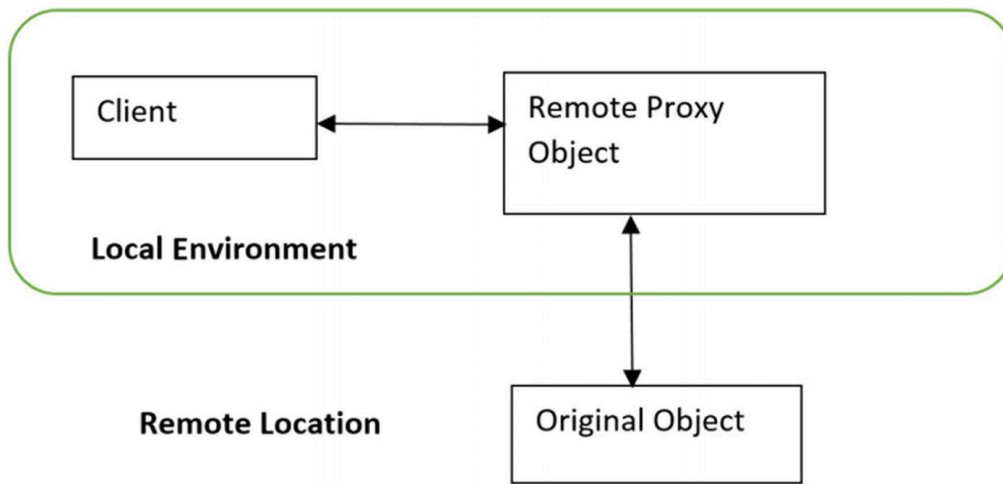


Figure 8-3 A simple remote proxy diagram

8.5 When can you use a virtual proxy?

They can be used to preserve some memory from being allocated to an object. If the actual object creation is an expensive operation, you can create a light copy of the intended object with the most important details and supply it to the user. The expensive object will be created only when it is truly needed. For example, you can use the concept to avoid loading an extremely large image unnecessarily for better application performance.

8.6 When can you use a protection proxy?

In an organization, the security team can implement a protection proxy to block internet access to specific websites. Demonstration 2 can help you understand this better.

Demonstration 2

Consider the following example, which is a modified version of the Proxy pattern implementation described earlier. For simplicity, let's assume you have only three registered users who can exercise the proxy method `doSomeWork(String user)`. You can see that this time this method accepts a `String` parameter. You use this parameter to validate a registered user. If an unwanted user (say `Robin`) tries to invoke the method, the system will reject this request. When the system rejects this kind of unwanted access, there is no point in making a proxy object. In the upcoming example, all registered users are initialized in the proxy class constructor.

Here's the complete program. You can find all parts of the program inside the package `jdp3e.proxy.implementation_2`.

```
// Subject.java
```

```
package jdp3e.proxy.implementation_2.components;
public abstract class Subject
{
    public abstract void doSomeWork(String user);
}
// ConcreteSubject.java
package jdp3e.proxy.implementation_2.components;
// ConcreteSubject class
class ConcreteSubject extends Subject
{
    @Override
    public void doSomeWork(String user)
    {
        System.out.println(user+ " invokes the method-doSomeWork().");
    }
}
// ProxySubject.java
package jdp3e.proxy.implementation_2.components;
import java.util.ArrayList;
import java.util.List;
/**
 * @author V.Sarcar
 * This is the proxy class.
 * It invokes the doSomeWork() from
 * the ConcreteSubject.
 */
public class ProxySubject extends Subject {
    private Subject subject;
    //String[] registeredUsers;
    String currentUser;
    List<String> registeredUsers;
    //Or, simply create this mutable list in one step
    //List<String> registeredUsers=new
    // ArrayList<String>(Arrays.asList( "Admin","Kate","Sam"));
    public ProxySubject () {
        // We need to create only one instance
        // of the ConcreteSubject.
        if (subject == null)      {

            subject= new ConcreteSubject();
        }
        // Initialize the registered users:
        registeredUsers = new ArrayList<String>();
        registeredUsers.add("Admin");
        registeredUsers.add("Kate");
        registeredUsers.add("Sam");
    }
}
```

```

@Override
public void doSomeWork(String user) {
    System.out.println("\n The proxy call is happening now.");
    System.out.println(user+" wants to invoke a proxy method.");
    if (registeredUsers.contains(user)){
        // Allow the registered user to invoke the method
        subject.doSomeWork(user);
    } else {
        System.out.println("Sorry, "+ user+ ",
                           you do not have access rights.");
    }
}
}
}
// Client.java
package jdp3e.proxy.implementation_2;
import jdp3e.proxy.implementation_2.components.Subject;
import jdp3e.proxy.implementation_2.components.ProxySubject;
/**
 * @author V.Sarcar

 * The client is talking to a ConcreteSubject instance
 * through a proxy method.
 */
class Client {
    public static void main(String[] args) {
        System.out.println("***Modified Proxy Pattern Demonstration.**
        // Admin is an authorized user
        Subject proxy = new ProxySubject();
        proxy.doSomeWork("Admin");
        // Robin is an unauthorized user
        proxy.doSomeWork("Robin");
    }
}

```

Output

Here's the modified output for you:

```

***Modified Proxy Pattern Demonstration.***
The proxy call is happening now.
Admin wants to invoke a proxy method.
Admin invokes the method-doSomeWork().
The proxy call is happening now.
Robin wants to invoke a proxy method.
Sorry, Robin, you do not have access rights.

```

8.7 It looks like proxies act like decorators (in Chapter 9). Is this correct?

Sometimes a proxy implementation can have similarities to a decorator, but you should not forget the true intent of a proxy. Decorators focus on adding responsibilities, whereas proxies focus on controlling access to an object. So, if you remember their purpose, in most cases you will be able to distinguish proxies from decorators.

Remember the following: In general, proxies work on the same interface but decorators can work on extended interfaces.

8.8 When should I think of designing a proxy?

As said earlier in the chapter, here are some important use cases where proxies can help you:

- You are writing test cases for a scenario that is still in the development phase or very hard to reproduce. For example, when you want to evaluate a particular behavior of an application that can be seen in a customer environment only, but you also recognize that when the application is running, the probability of getting the behavior is very low. In such a case, you can mimic the customer environment behavior in your proxy object and execute your test cases to evaluate this behavior.
- You want to hide the complexity and enhance the security of the system.

8.9 What are the cons associated with proxies?

Here are some factors that you should keep in mind while using this pattern:

- The overall response time can be an issue because you are not directly talking to the actual object.
- You need to maintain an additional layer for the proxies.
- Lastly, a proxy can hide the actual responses from objects. For example, consider the case when a proxy method modifies the original exception thrown by a method. Debugging and fixing such a case is not straightforward.

Previous chapter

[< 7. Singleton Pattern](#)

Next chapter

[9. Decorator Pattern >](#)